VIDYAVARDHAKA COLLEGE OF ENGINEERING

(Autonomous, affiliated to VTU)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Activity-Based Assessment Report

ON

CAMPUS ATTENDANCE TREND ANALYTICS USING APACHE HIVE ON THE HADOOP ECOSYSTEM

Submitted by:
4VV23CS033 Bharath Krishna K K
4VV23CS042 Charan P
4VV23CS006 Abu Safwan

Course Name: Big Data Analytics
Course Code: BCSBD713
Academic Year: 2026-27

Sample report: replace placeholders with actual implementation evidence and results.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Context

2. PROBLEM STATEMENT

3. SIGNIFICANCE OF THE PROBLEM

4. OBJECTIVES

5. SCOPE OF THE PROJECT

6. EXISTING SYSTEM AND LIMITATIONS

7. PROPOSED SYSTEM

8. HADOOP ECOSYSTEM AND TOOL SELECTION

9. SYSTEM REQUIREMENTS

10. DATASET DESCRIPTION

10.1 SAMPLE RECORDS

10.2 DATASET CAVEAT

11. SYSTEM ARCHITECTURE

11.1 DATA FLOW

12. METHODOLOGY

13. IMPLEMENTATION

14. HIVE ANALYTICS ALGORITHM AND DATA FLOW

15. APPLICATION DEMONSTRATION

16. RESULTS AND ANALYSIS

16.1 INTERPRETATION

17. ADVANTAGES

18. LIMITATIONS

19. FUTURE ENHANCEMENTS

20. CONCLUSION

21. REFERENCES

1. INTRODUCTION

Big Data Analytics focuses on extracting useful information from datasets whose volume, variety, or processing requirements make traditional approaches less effective. Educational institutions generate attendance records repeatedly across students, subjects, sections, semesters, and dates. Analysing these records manually can become repetitive and difficult when the dataset grows.

This project demonstrates how Apache Hadoop can be used to store attendance records in HDFS and how Apache Hive can provide SQL-like batch analytics over the stored data. Rather than attempting to build a complete college attendance management system, the project focuses on transforming structured attendance observations into subject-wise, student-wise, section-wise and time-based summaries.

The selected Hadoop technologies are HDFS and Apache Hive. HDFS represents the storage layer, while Hive provides a query layer over the stored data. The prototype is designed as an academic demonstration and uses synthetic/anonymized data rather than real student personal information.

1.1 Project Context

Attendance records may be generated repeatedly for different subjects and dates. Aggregating these observations can reveal overall attendance percentage, subject-level differences, low-attendance cases, and trends across departments or sections.

The project demonstrates a complete Hadoop ecosystem workflow: data preparation, HDFS storage, Hive table creation, HiveQL analysis, result inspection, visualization, and interpretation.

2. PROBLEM STATEMENT

How can attendance records be stored and analysed efficiently using a Hadoop ecosystem tool to identify attendance trends, subject-wise performance, and low-attendance cases?

2.1 Problem Description
Attendance information may be distributed across spreadsheets or exported files. Manual calculation of percentages and comparisons becomes repetitive as the number of students, subjects and class sessions increases.

2.2 Expected Solution
The proposed solution stores the attendance dataset in HDFS, exposes it through an Apache Hive external table, and uses HiveQL queries to calculate summary metrics. The query results are then inspected and visualized for academic interpretation.

3. SIGNIFICANCE OF THE PROBLEM

Attendance analysis has practical significance because institutions need to monitor student participation and identify cases requiring attention.

Operational significance: faculty can identify subjects, sections or students with low attendance.
Analytical significance: repeated records can be converted into comparable metrics.
Scalability significance: the same Hive query pattern can be applied to substantially larger files.
Educational significance: the project demonstrates HDFS and Hive in a complete Big Data Analytics workflow.

3.1 Why Big Data Technology?
A small CSV can be processed using conventional Python or spreadsheet tools. However, the ABA requires a Hadoop ecosystem solution. Hadoop and Hive provide a suitable academic model for demonstrating distributed storage and SQL-like analytics over large structured datasets.

4. OBJECTIVES

- Identify a real-world educational analytics problem suitable for Hadoop.
- Store attendance observations in HDFS.
- Create an Apache Hive external table over the attendance data.
- Use HiveQL to calculate attendance percentages and summary statistics.
- Identify low-attendance student/subject combinations.
- Compare attendance across subjects, departments and sections.

• Demonstrate the complete input-to-output Hadoop/Hive workflow.
• Present analytical results using tables and charts.

5. SCOPE OF THE PROJECT

The scope is limited to batch analysis of structured attendance records. The prototype does not replace an institutional attendance system, send official notices, or expose real student personal information.

5.1 Included
CSV dataset preparation; HDFS storage; Hive table creation; HiveQL batch analytics; result visualization; command-line demonstration.

5.2 Not Included
Real-time attendance capture; biometric integration; production student information system; predictive ML model; automated institutional notifications; processing of identifiable student data.

6. EXISTING SYSTEM AND LIMITATIONS

A conventional approach would load an attendance CSV into a local program or spreadsheet and calculate percentages manually. This is simple for small datasets, but it does not demonstrate distributed storage or Hadoop-based analytics.

Local CSV/Python script — simple and fast for small data — limited distributed processing.
Spreadsheet analysis — easy to inspect — repetitive and difficult to scale.
Hadoop + Hive — HDFS storage and SQL-like batch analytics — requires environment setup and operational knowledge.

7. PROPOSED SYSTEM

The proposed system uses a Hadoop pipeline in which the attendance CSV is placed in HDFS, registered as an external Hive table, queried using HiveQL, and transformed into analytical results.

Major Components
HDFS Input Layer: stores the attendance CSV.
Hive External Table: provides a schema over the HDFS file.
HiveQL Analytics Layer: performs grouping, counting, conditional aggregation and percentage calculations.
Result Layer: stores/displays query output for analysis.
Visualization Layer: converts selected results into charts and tables for interpretation.

8. HADOOP ECOSYSTEM AND TOOL SELECTION

8.1 Apache Hadoop
Apache Hadoop is selected because the assignment requires a Hadoop ecosystem tool and because HDFS provides a clear distributed storage foundation.

8.2 HDFS
Hadoop Distributed File System provides storage for the input attendance file and can distribute file blocks across nodes in a cluster.

8.3 Apache Hive
Hive is selected as the primary analytics tool. It provides a SQL-like query language, HiveQL, making aggregation tasks accessible while demonstrating analytics on Hadoop.

8.4 HiveQL
HiveQL is used for subject-wise attendance, low-attendance analysis, section comparison and time-based aggregation.

9. SYSTEM REQUIREMENTS

9.1 Hardware Requirements
A laptop/desktop with at least 8 GB RAM is recommended for a comfortable local academic Hadoop environment. A

dual-core or better processor is suitable. At least 10–20 GB free storage is recommended for Hadoop, Hive, logs, datasets and outputs.

9.2 Software Requirements
Linux, WSL2 or another supported Hadoop environment; Apache Hadoop 3.x; Apache Hive; Java runtime required by the selected Hadoop/Hive versions; Python 3.x optionally for dataset generation and charts; terminal/shell; spreadsheet or presentation software.

9.3 Suggested Project Folder
campus_attendance_hive/
data/attendance.csv
hive/create_table.sql
hive/analytics.sql
scripts/generate_dataset.py
scripts/make_charts.py
results/
charts/
screenshots/
README.md

10. DATASET DESCRIPTION

The project uses a synthetic/anonymized attendance dataset intended for reproducible classroom demonstration. It should not be presented as official college attendance evidence.

Suggested fields:
student_id — anonymized student identifier
department — department label
semester — semester number
section — section label
subject_code — subject identifier
subject_name — subject name
attendance_date — class date
period_number — class period
status — PRESENT or ABSENT

A practical classroom dataset can contain approximately 5,000–20,000 records. The exact final record count should be reported after the implementation dataset is generated.

10.1 SAMPLE RECORDS

Example:
S001,CSE,7,A,BCSBD713,Big Data Analytics,2026-10-01,1,PRESENT
S001,CSE,7,A,BCSBD713,Big Data Analytics,2026-10-02,1,ABSENT
S002,CSE,7,A,BCSBD713,Big Data Analytics,2026-10-01,1,PRESENT

These are sample-format records only. The final report should include the actual generated dataset characteristics.

10.2 DATASET CAVEAT

The dataset is synthetic/anonymized. It is not a verified institutional attendance database and should not be presented as actual student performance evidence. If real data is used later, personal identifiers should be removed or protected and institutional authorization should be obtained.

11. SYSTEM ARCHITECTURE

Attendance CSV → HDFS → Hive External Table → HiveQL Queries → Aggregated Results → Charts/Tables → Interpretation

The external table allows Hive to query the data stored in HDFS without requiring a separate application database. The same analytical workflow can be extended to larger datasets in a properly configured Hadoop environment.

11.1 DATA FLOW

CSV file → HDFS input directory
HDFS file → Hive external table
HiveQL → GROUP BY / COUNT / SUM / CASE / HAVING
Query result → CSV/text output
Output → chart/table
Chart/table → interpretation

12. METHODOLOGY

Step 1 – Prepare the dataset: generate or validate the attendance CSV and check field names, dates and status values.

Step 2 – Load data into HDFS: create an HDFS input directory and upload the CSV file.

Step 3 – Create Hive table: create an external Hive table with the attendance schema.

Step 4 – Query: run HiveQL queries for overall, subject-wise, student-wise, section-wise and time-based analysis.

Step 5 – Validate: compare selected query results with independent calculations or sample records.

Step 6 – Visualize: create charts for subject comparison, attendance distribution and time trends.

Step 7 – Interpret: explain the major trends without claiming unsupported conclusions.

13. IMPLEMENTATION

13.1 Hive Table Logic
The Hive external table defines the attendance schema and points to the HDFS directory containing the CSV data.

13.2 Subject-wise Analytics
The query counts all records, counts PRESENT records using conditional aggregation, and calculates attendance percentage.

13.3 Low Attendance Analytics
The query groups records by student and subject and identifies groups below the selected threshold.

13.4 Representative HiveQL

```
SELECT subject_code, subject_name,
COUNT(*) AS total_classes,
SUM(CASE WHEN status='PRESENT' THEN 1 ELSE 0 END) AS present_count,
ROUND(100.0*SUM(CASE WHEN status='PRESENT' THEN 1 ELSE 0 END)/COUNT(*),2) AS attendance_pct
FROM attendance
GROUP BY subject_code, subject_name;
```

The final report should include the exact commands and query outputs produced during implementation.

14. HIVE ANALYTICS ALGORITHM AND DATA FLOW

For each attendance record, Hive reads the structured fields from the external table. GROUP BY operations form logical groups such as subject or student-subject. COUNT calculates class observations, while conditional SUM counts PRESENT observations. Attendance percentage is present observations divided by total observations multiplied by 100.

Low-attendance analysis uses an aggregate query and filters groups below the selected threshold, such as 75%. The threshold is configurable and should be aligned with the institution's required policy if the project is later deployed.

15. APPLICATION DEMONSTRATION

15.1 Demonstration Sequence
1. Open the project folder and show attendance.csv.
2. Explain the dataset columns.
3. Start Hadoop/HDFS and verify availability.

4. Create the HDFS input directory.
5. Upload the CSV file.
6. Open Hive and create/use the project database.
7. Create the external attendance table.
8. Run a subject-wise HiveQL query.
9. Run the low-attendance query.
10. Show the output and one chart.
11. Explain one or two actual insights.

15.2 What to Explain
HDFS is the storage layer. Hive provides SQL-like analytics. GROUP BY creates analytical groups. Conditional aggregation calculates PRESENT counts. The final values are descriptive summaries of the supplied dataset, not predictions.

16. RESULTS AND ANALYSIS

The final report must contain actual execution results. Do not invent numerical values before the Hive implementation is completed.

Recommended result items:
• Total attendance records processed: [ACTUAL VALUE]
• Overall attendance percentage: [ACTUAL VALUE]
• Highest-attendance subject: [ACTUAL VALUE]
• Lowest-attendance subject: [ACTUAL VALUE]
• Number of low-attendance student-subject groups: [ACTUAL VALUE]
• Department/section comparison: [ACTUAL VALUE]
• Weekly/date-based trend: [ACTUAL VALUE]

Recommended evidence:
Figure 1 – HDFS input directory screenshot
Figure 2 – Hive table creation/SHOW TABLES screenshot
Figure 3 – Subject-wise query and output
Figure 4 – Low-attendance query and output
Figure 5 – Subject-wise attendance chart
Figure 6 – Attendance distribution/trend chart

16.1 INTERPRETATION

Interpretation should be written only after actual results are available. Explain which subjects or groups show higher/lower attendance, whether any low-attendance concentration is visible, and what the trend means within the limitations of the synthetic/anonymized dataset.

17. ADVANTAGES

• Demonstrates HDFS and Apache Hive in one application.
• Uses a clear real-world educational analytics problem.
• HiveQL is accessible and easy to explain.
• Supports multiple analytical views from the same dataset.
• Results can be visualized using simple charts.
• The design can be extended to larger datasets and additional Hadoop tools.

18. LIMITATIONS

• The demonstration dataset is synthetic/anonymized.
• The prototype is primarily batch-oriented.
• A local/single-node environment does not demonstrate the full performance of a production cluster.
• Data quality directly affects the accuracy of analytics.
• The project is descriptive rather than predictive.
• Production deployment would require privacy, security, governance and integration controls.

19. FUTURE ENHANCEMENTS

• Add an interactive attendance dashboard.
• Automate attendance-file ingestion into HDFS.
• Add semester-to-semester comparison.
• Generate configurable low-attendance alerts.
• Integrate timetable or institutional attendance sources.
• Explore Apache Spark for advanced/iterative analytics.
• Add predictive attendance-risk modelling.
• Deploy on a multi-node or cloud Hadoop environment.
• Add data-quality validation and audit logging.

20. CONCLUSION

This project demonstrates how a real-world attendance analytics problem can be addressed using the Hadoop ecosystem. Attendance records are stored in HDFS and analysed through Apache Hive using HiveQL. The workflow converts raw attendance observations into subject-wise, student-wise, section-wise and time-based summaries.

The project demonstrates an end-to-end Big Data workflow: problem identification, dataset preparation, HDFS storage, Hive table creation, analytical querying, result generation and interpretation. Although the academic prototype uses synthetic/anonymized data, the same approach can be extended to larger datasets with appropriate privacy, governance and deployment controls.

21. REFERENCES

1. Apache Hadoop Documentation – HDFS and Hadoop ecosystem concepts.
2. Apache Hive Documentation – Hive overview and language reference.
3. Apache Hive Language Manual – HiveQL syntax and analytical operations.
4. Apache Software Foundation – Apache Hadoop project documentation.
5. Apache Software Foundation – Apache Hive project documentation.
6. Course material and classroom notes on Big Data Analytics and Hadoop ecosystem.